Controlled AI/ML Production Engineering: Responsible Software Development with Agentic AI

Iain J. Clark

Document Controls
Version: 1.0 25 April 2026
Status: Working Draft

Overview

Agentic AI tools such as GitHub Copilot, Cursor, and Claude Code can significantly accelerate software development for AI/ML production systems. Used well, they can support the rapid translation of end-user requirements into code generation, testing, refactoring, and documentation. Used poorly, they can introduce hidden defects and compromise quality. This note sets out a practical framework for the responsible use of Agentic AI in software construction; the single most important part of this framework being the key principle: **final accountability persists** with the human who uses and supervises these tools. This framework is illustrative and represents one possible approach; it is not tied to any specific organisational implementation.

1. Introduction

The supervising developer remains accountable for all AI-assisted code.

---

2. Operational Guidelines

The human developer remains at the centre of the development workflow and supervises the process. AI agents may propose, draft, critique, test, and implement, but they do not approve requirements, accept design risk, or certify production readiness.

2.1. Agent Roles

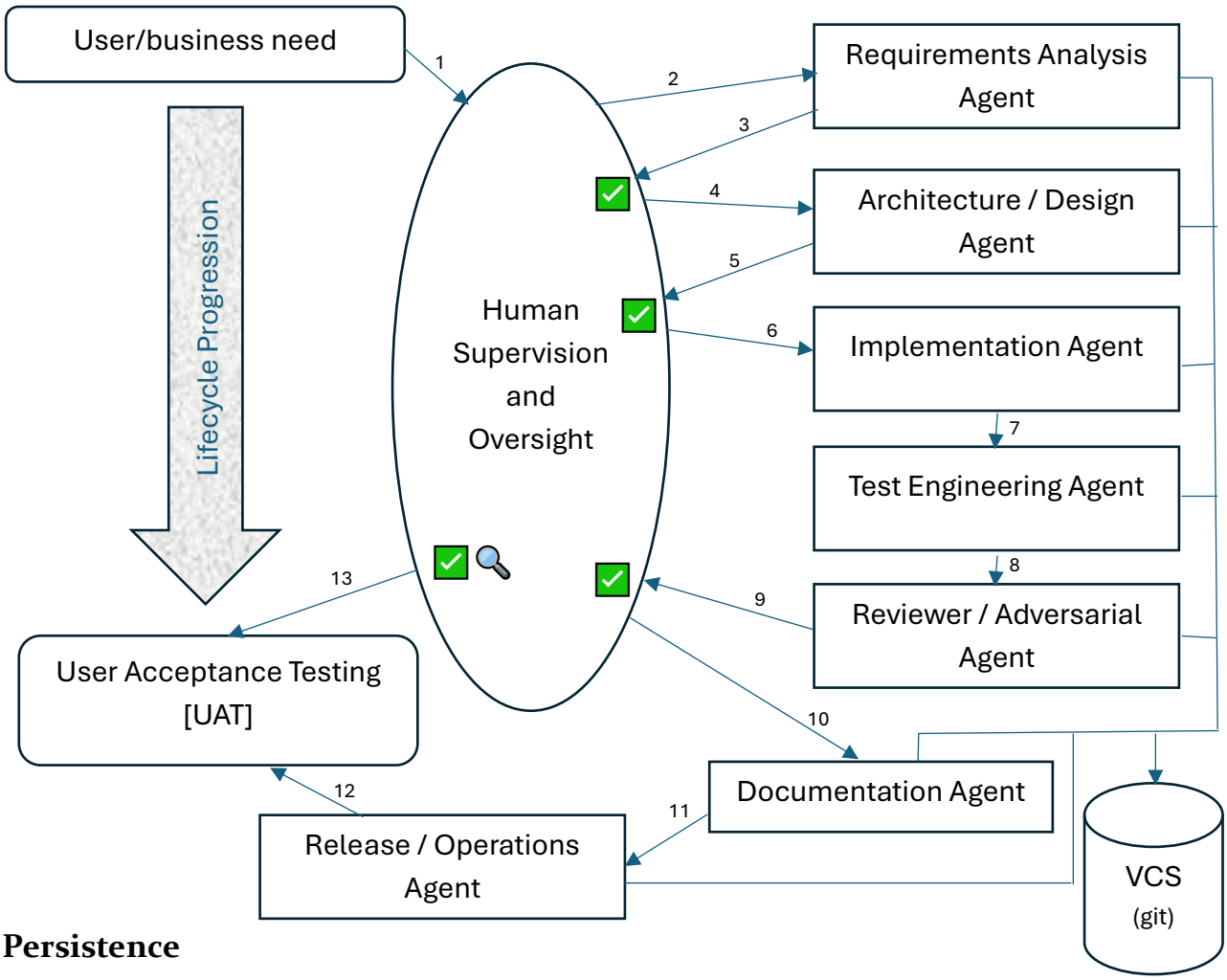
One possible framework involves the separation of Agentic AI assistance into well-defined roles, each of which is bounded by human intent and supervised by the human developer, e.g.

Agent Role	Purpose	Example Tools (2026)
Requirements Analysis Agent	Convert user needs into structured requirements and acceptance criteria	ChatGPT / Claude
Design/Architecture Agent	Propose system structure, interfaces, data flows, dependencies	ChatGPT / Claude
Research Agent	Examine literature and suggest current references	Perplexity
Implementation Agent	Generate or modify code under approved requirements	Cursor / Claude Code / GitHub Copilot
Test Engineering Agent	Produce unit, integration, regression, and edge-case tests	Cursor / Claude Code / ChatGPT

Reviewer/Adversarial Agent	Challenge assumptions, find defects, identify risks	Claude / ChatGPT
Documentation Agent	Maintain README, runbooks, diagrams, decision records	ChatGPT / Claude
Release/Operations Agent	Support deployment notes, monitoring checks, and rollback plans	ChatGPT / Claude (with operational delegation to Cursor / Claude Code)

2.2. Controlled Workflow

The interaction between the developer-engineer, stakeholders, and agentic AI proceeds as:



2.3. Persistence

The architecture shown is a Directed Acyclic Graph [DAG] and should be committed to a version control system, e.g. Git, to log both the code generated, the markdown artifacts generated by each agent, and the inter-agent transactions that generate code.

3. Case Study

See www.github.com/iainjclark/petrol-price-prediction/ for a repo built using this.

The supervising developer remains accountable for all AI-assisted code.